Hoare Logic I

Triples and rules

Cristina David

Lecture 8

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

Where we have been

- **Statics**. Typing judgements $\vdash e : \sigma$.
- **Dynamics**. Transitions $e \mapsto e'$ and $e \mapsto^* e'$.
- **Type safety**. Progress and preservation.

Where we are going

Type safety says: well-typed programs do not get stuck.

That is a real theorem, and your first verification result. But what does it *not* say?

Where we are going

Type safety says: well-typed programs do not get stuck.

That is a real theorem, and your first verification result. But what does it *not* say?

$\mathsf{Int} \to \mathsf{Int}$ is inhabited by:

- the doubling function
- the constant 0
- the function computing the nth prime

Type safety cannot tell them apart.

Where we are going

Type safety says: well-typed programs do not get stuck.

That is a real theorem, and your first verification result. But what does it *not* say?

$\mathsf{Int} \to \mathsf{Int}$ is inhabited by:

- the doubling function
- the constant 0
- the function computing the nth prime

Type safety cannot tell them apart.

Today: a third semantics that can.

The gap

Statics: a *coarse* specification

It says a program will not go wrong. Never which function it computes.

The gap

Statics: a *coarse* specification

It says a program will not go wrong. Never which function it computes.

Dynamics: not a specification at all

The *ground truth* of execution. It says what *happens*,
not what should happen.

The gap

Statics: a *coarse* specification

It says a program will not go wrong. Never which function it computes.

Dynamics: not a specification at all

The *ground truth* of execution. It says what *happens*,
not what should happen.

So

We cannot *state* what a program is supposed to compute,
and therefore cannot *prove* that it does.

Today

Axiomatic semantics gives a program's meaning by the logical assertions that hold about it. Hoare logic is an instance of axiomatic semantics.

The language: Imp

We change vehicles

The λ-calculus hid the thing we now care about: mutable state.

Hoare logic reasons about how state changes. So it needs a language with some state-changing, effectful construct for the reasoning to bite on.

We change vehicles

The λ-calculus hid the thing we now care about: mutable state.

Hoare logic reasons about how state changes. So it needs a language with some state-changing, effectful construct for the reasoning to bite on.

Imp — the minimal imperative language.

Everything to the end of the unit lives in Imp (plus, later, a heap).

Imp: syntax

$$a ::= n \mid x \mid a + a \mid a - a \mid a * a$$ arithmetic expressions

$$b ::= \text{true} \mid \text{false} \mid a = a \mid a \leq a \mid \neg b \mid b \wedge b$$ boolean expressions

$$\begin{aligned} c ::= & \ \text{skip} \\ & \mid x := a \\ & \mid c \,;\, c \\ & \mid \text{if } b \text{ then } c \text{ else } c \\ & \mid \text{while } b \text{ do } c \end{aligned}$$ commands

Imp: states

A state is a total map from variables to integers:

$$\sigma : \mathrm{Var} \to \mathbb{Z}$$

Expressions denote functions of the state:

$$[\![a]\!]\sigma \in \mathbb{Z} \qquad\qquad [\![b]\!]\sigma \in \{tt, ff\}$$

Update notation: $\sigma[x \mapsto v]$ is σ with x remapped to v.

Variables hold integers only — a boolean cannot live in the state. Assignment is $x := a$, and booleans appear only as guards, evaluated on demand and never stored.

A notation warning

σ changes meaning today

Previously, σ ranged over types: $\vdash e : \sigma$.

From now on, σ ranges over states.

This is the standard convention in the verification literature.

Hoare Triples

$$\{P\}\ c\ \{Q\}$$

The Hoare triple

$$\{P\}\ c\ \{Q\}$$

Reading

If P holds before executing c,
and c terminates,
then Q holds afterwards.

The Hoare triple

$$\{P\}\ c\ \{Q\}$$

Reading

If P holds before executing c,
and c terminates,
then Q holds afterwards.

P is the precondition. Q is the postcondition.

This is called partial correctness. It says nothing about whether c terminates.

Logical assertions

An assertion P is a predicate on states — a first-order formula over the program variables.

Examples:

$$x > 0 \qquad x = y + 1 \vee y = 0 \qquad \exists k.\ x = 2k \qquad \text{True} \qquad \text{False}$$

Write $\sigma \vDash P$ for "P holds in state σ".

Logical assertions

An assertion P is a predicate on states — a first-order formula over the program variables.

Examples:

$$x > 0 \qquad x = y + 1 \vee y = 0 \qquad \exists k.\ x = 2k \qquad \text{True} \qquad \text{False}$$

Write $\sigma \vDash P$ for "P holds in state σ".

Note

Assertions are *not* boolean expressions of Imp. They may use quantifiers, they are the language of *mathematics*, not of the program.

Quick check

Which of these are valid?

1. $\{\text{True}\}\ \ x := 5\ \ \{x = 5\}$
2. $\{\text{True}\}\ \ x := 5\ \ \{x = 6\}$
3. $\{x = 6\}\ \ x := 5\ \ \{x = 5\}$

Quick check

Which of these are valid?

1. $\{\text{True}\}\ \ x := 5\ \ \{x = 5\}$
2. $\{\text{True}\}\ \ x := 5\ \ \{x = 6\}$
3. $\{x = 6\}\ \ x := 5\ \ \{x = 5\}$

Yes / No / Yes.

The rules

The proof system

We now define a judgement $\vdash \{P\}\ c\ \{Q\}$, read "derivable": the triple can be derived using the rules below.

Six rules. One per syntactic form, plus one more.

Skip and sequencing

$$\frac{}{\vdash \{P\}\ \mathsf{skip}\ \{P\}}\ \text{Skip}$$

No premises, so this is an axiom.

Skip and sequencing

$$\frac{}{\vdash \{P\}\ \mathsf{skip}\ \{P\}}\ \text{Skip}$$

No premises, so this is an axiom.

$$\frac{\vdash \{P\}\ c_1\ \{R\} \qquad \vdash \{R\}\ c_2\ \{Q\}}{\vdash \{P\}\ c_1; c_2\ \{Q\}}\ \text{Seq}$$

The midpoint assertion R is the glue. Finding it is the whole work of a sequential proof — and, as we will see, the assignment axiom will *compute* it for us.

Conditionals

$$\frac{\vdash \{P \wedge b\}\ c_1\ \{Q\} \qquad \vdash \{P \wedge \neg b\}\ c_2\ \{Q\}}{\vdash \{P\}\ \text{if}\ b\ \text{then}\ c_1\ \text{else}\ c_2\ \{Q\}}\ \text{If}$$

Each branch may assume the guard's truth value.

Both branches must establish the same Q.

The rule merges the two branches back together.

Assignment — the surprising one

$$\frac{}{\vdash \{Q[a/x]\}\ x := a\ \{Q\}}\ \text{Assign}$$

Assignment — the surprising one

$$\frac{}{\vdash \{Q[a/x]\}\ x := a\ \{Q\}}\ \text{Assign}$$

Read it right to left.

To guarantee Q *after* $x := a$,
demand that Q-with-a-in-place-of-x holds *before*.

Assignment: micro-examples

Want	Command	Axiom gives
$x > 0$	$x := x + 1$	$x + 1 > 0$, i.e. $x > -1$
$x = 5$	$x := 5$	$5 = 5$, i.e. True
$x = y$	$x := y$	$y = y$, i.e. True
$x = 6$	$x := 5$	$5 = 6$, i.e. False

Assignment: micro-examples

Want	Command	Axiom gives
$x > 0$	$x := x + 1$	$x + 1 > 0$, i.e. $x > -1$
$x = 5$	$x := 5$	$5 = 5$, i.e. True
$x = y$	$x := y$	$y = y$, i.e. True
$x = 6$	$x := 5$	$5 = 6$, i.e. False

The last one is the sanity check.

You cannot land in $x = 6$ by assigning 5. The axiom gives you False, the only precondition from which the triple holds, and one you can never satisfy.

Why not a forward rule?

You may have expected something like:

$$\{P\}\ \ x := a\ \ \{P \wedge x = a\} \qquad \text{(wrong)}$$

Why not a forward rule?

You may have expected something like:

$$\{P\}\ \ x := a\ \ \{P \wedge x = a\} \qquad \text{(wrong)}$$

Try it: $\{x = 0\}\ \ x := x + 1\ \ \{x = 0 \wedge x = x + 1\}$

The postcondition is unsatisfiable. The old value of x is *gone*, but P still refers to it.

Why not a forward rule?

You may have expected something like:

$$\{P\}\ \ x := a\ \ \{P \wedge x = a\} \qquad \text{(wrong)}$$

Try it: $\{x = 0\}\ \ x := x + 1\ \ \{x = 0 \wedge x = x + 1\}$

The postcondition is unsatisfiable. The old value of x is *gone*, but P still refers to it.

A correct forward rule must invent a name x' for the old value:

$$\{P\}\ \ x := a\ \ \{\exists x'.\ P[x'/x]\ \wedge\ x = a[x'/x]\}$$

Why not a forward rule?

You may have expected something like:

$$\{P\}\ \ x := a\ \ \{P \wedge x = a\} \qquad \text{(wrong)}$$

Try it: $\{x = 0\}\ \ x := x + 1\ \ \{x = 0 \wedge x = x + 1\}$

The postcondition is unsatisfiable. The old value of x is *gone*, but P still refers to it.

A correct forward rule must invent a name x' for the old value:

$$\{P\}\ \ x := a\ \ \{\exists x'.\ P[x'/x]\ \wedge\ x = a[x'/x]\}$$

On the example: $\exists x'.\ x' = 0 \wedge x = x' + 1$, i.e. $x = 1$. Correct, but

existential quantifiers accumulate with every assignment. The backward axiom sidesteps this entirely.

Loops — the invariant rule

$$\frac{\vdash \{I \wedge b\}\ c\ \{I\}}{\vdash \{I\}\ \text{while } b \text{ do } c\ \{I \wedge \neg b\}}\ \text{While}$$

I is the loop invariant:

- true on entry,
- preserved by the body (that is the premise),
- hence true on exit — and on exit, the guard has failed.

Loops — the invariant rule

$$\frac{\vdash \{I \wedge b\}\ c\ \{I\}}{\vdash \{I\}\ \text{while } b \text{ do } c\ \{I \wedge \neg b\}}\ \text{While}$$

I is the loop invariant:

- true on entry,
- preserved by the body (that is the premise),
- hence true on exit — and on exit, the guard has failed.

Where is I from?

The rule does not tell you. Finding I is the creative core of verification, and it is the whole of next lecture.

Consequence — where logic enters

$$\frac{P \Rightarrow P' \qquad \vdash \{P'\}\ c\ \{Q'\} \qquad Q' \Rightarrow Q}{\vdash \{P\}\ c\ \{Q\}}\ \text{Conseq}$$

The other five rules are syntax-directed: look at c, apply the rule.

This one is not. It can be applied anywhere, any time.

Consequence — where logic enters

$$\frac{P \Rightarrow P' \qquad \vdash \{P'\}\ c\ \{Q'\} \qquad Q' \Rightarrow Q}{\vdash \{P\}\ c\ \{Q\}}\ \text{Conseq}$$

The other five rules are syntax-directed: look at c, apply the rule.

This one is not. It can be applied anywhere, any time. Its premises $P \Rightarrow P'$ and $Q' \Rightarrow Q$ are entailments in the assertion logic.

This is the seam between program reasoning and logical reasoning.

Consequence: mind the direction

$$P \Rightarrow P' \qquad\qquad Q' \Rightarrow Q$$

Strengthen the precondition. Weaken the postcondition.

Consequence: mind the direction

$$P \Rightarrow P' \qquad\qquad Q' \Rightarrow Q$$

Strengthen the precondition. Weaken the postcondition.

You may always assume more and promise less.

You may never assume less and promise more.

The six rules, together

$$\frac{}{\{P\}\ \text{skip}\ \{P\}}\ \text{Skip} \qquad \frac{}{\{Q[a/x]\}\ x := a\ \{Q\}}\ \text{Assign}$$

$$\frac{\{P\}\ c_1\ \{R\} \qquad \{R\}\ c_2\ \{Q\}}{\{P\}\ c_1; c_2\ \{Q\}}\ \text{Seq} \qquad \frac{\{P \wedge b\}\ c_1\ \{Q\} \qquad \{P \wedge \neg b\}\ c_2\ \{Q\}}{\{P\}\ \text{if}\ b\ \text{then}\ c_1\ \text{else}\ c_2\ \{Q\}}\ \text{If}$$

$$\frac{\{I \wedge b\}\ c\ \{I\}}{\{I\}\ \text{while}\ b\ \text{do}\ c\ \{I \wedge \neg b\}}\ \text{While} \qquad \frac{P \Rightarrow P' \qquad \{P'\}\ c\ \{Q'\} \qquad Q' \Rightarrow Q}{\{P\}\ c\ \{Q\}}\ \text{Conseq}$$

That is the whole logic.

A worked example

The swap

$$\{x = A \wedge y = B\} \;\; t := x; \;\; x := y; \;\; y := t \;\; \{x = B \wedge y = A\}$$

A and B are logical variables (auxiliary variables).

- They do not appear in the program.
- They are universally quantified constants that name the initial values.
- Without them we could not *state* what swapping means.

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\text{3rd:} \quad y := t \quad \text{pre} = Q[t/y] \qquad = (x = B \wedge t = A)$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\begin{array}{lll} \text{3rd:} \quad y := t & \text{pre} = Q[t/y] & = (x = B \wedge t = A) \\ \text{2nd:} \quad x := y & \text{pre} = (\cdots)[y/x] & = (y = B \wedge t = A) \end{array}$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\begin{array}{lllll} \text{3rd:} & y := t & \text{pre} = Q[t/y] & = (x = B \wedge t = A) \\ \text{2nd:} & x := y & \text{pre} = (\cdots)[y/x] & = (y = B \wedge t = A) \\ \text{1st:} & t := x & \text{pre} = (\cdots)[x/t] & = (y = B \wedge x = A) \end{array}$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\begin{array}{lll} \text{3rd:} \quad y := t & \text{pre} = Q[t/y] & = (x = B \wedge t = A) \\ \text{2nd:} \quad x := y & \text{pre} = (\cdots)[y/x] & = (y = B \wedge t = A) \\ \text{1st:} \quad t := x & \text{pre} = (\cdots)[x/t] & = (y = B \wedge x = A) \end{array}$$

By three Assign axioms and two Seq steps:

$$\vdash \{y = B \wedge x = A\} \;\; t := x;\; x := y;\; y := t \;\; \{x = B \wedge y = A\}$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\begin{aligned}
&\text{3rd:} \quad y := t \quad \text{pre} = Q[t/y] && = (x = B \wedge t = A) \\
&\text{2nd:} \quad x := y \quad \text{pre} = (\cdots)[y/x] && = (y = B \wedge t = A) \\
&\text{1st:} \quad t := x \quad \text{pre} = (\cdots)[x/t] && = (y = B \wedge x = A)
\end{aligned}$$

By three Assign axioms and two Seq steps:

$$\vdash \{y = B \wedge x = A\} \;\; t := x;\; x := y;\; y := t \;\; \{x = B \wedge y = A\}$$

And Conseq closes it: $(x = A \wedge y = B) \Rightarrow (y = B \wedge x = A)$. ■

Correctness of the logic

Small-step, or big-step?

Previously, the dynamics was small-step:

$$e \mapsto e' \qquad\qquad e \mapsto^* e'$$

Today it will be big-step:

$$\langle c, \sigma \rangle \Downarrow \sigma'$$

straight from a command to its *final state*, saying nothing about the way there.

Small-step, or big-step?

Previously, the dynamics was small-step:

$$e \mapsto e' \qquad\qquad e \mapsto^* e'$$

Today it will be big-step:

$$\langle c, \sigma \rangle \Downarrow \sigma'$$

straight from a command to its *final state*, saying nothing about the way there.

They agree: $\langle c, \sigma \rangle \Downarrow \sigma'$ iff $\langle c, \sigma \rangle \mapsto^* \langle \mathsf{skip}, \sigma' \rangle$

Small-step, or big-step?

Previously, the dynamics was small-step:

$$e \mapsto e' \qquad\qquad e \mapsto^* e'$$

Today it will be big-step:

$$\langle c, \sigma \rangle \Downarrow \sigma'$$

straight from a command to its *final state*, saying nothing about the way there.

They agree: $\langle c, \sigma \rangle \Downarrow \sigma'$ iff $\langle c, \sigma \rangle \mapsto^* \langle \mathsf{skip}, \sigma' \rangle$

Why big-step? Because a Hoare specification constrains only the initial and final states, and that is exactly the relation $\Downarrow$ gives us.

Imp: dynamics (big-step)

$$\frac{}{\langle \mathsf{skip},\, \sigma\rangle \Downarrow \sigma} \qquad \frac{}{\langle x := a,\, \sigma\rangle \Downarrow \sigma[x \mapsto [\![a]\!]\sigma]} \qquad \frac{\langle c_1,\, \sigma\rangle \Downarrow \sigma'' \qquad \langle c_2,\, \sigma''\rangle \Downarrow \sigma'}{\langle c_1; c_2,\, \sigma\rangle \Downarrow \sigma'}$$

$$\frac{[\![b]\!]\sigma = tt \qquad \langle c_1,\, \sigma\rangle \Downarrow \sigma'}{\langle \mathsf{if}\ b\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2,\, \sigma\rangle \Downarrow \sigma'} \qquad \frac{[\![b]\!]\sigma = ff \qquad \langle c_2,\, \sigma\rangle \Downarrow \sigma'}{\langle \mathsf{if}\ b\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2,\, \sigma\rangle \Downarrow \sigma'}$$

$$\frac{[\![b]\!]\sigma = ff}{\langle \mathsf{while}\ b\ \mathsf{do}\ c,\, \sigma\rangle \Downarrow \sigma} \qquad \frac{[\![b]\!]\sigma = tt \qquad \langle c,\, \sigma\rangle \Downarrow \sigma'' \qquad \langle \mathsf{while}\ b\ \mathsf{do}\ c,\, \sigma''\rangle \Downarrow \sigma'}{\langle \mathsf{while}\ b\ \mathsf{do}\ c,\, \sigma\rangle \Downarrow \sigma'}$$

Valid Hoare triple

So far "valid" was informal. With $\Downarrow$ we can now define it:

Validity

$\vDash \{P\}\ c\ \{Q\}$ iff for all σ, σ':

$$\sigma \vDash P \quad \text{and} \quad \langle c, \sigma \rangle \Downarrow \sigma' \qquad \text{implies} \qquad \sigma' \vDash Q.$$

Do the rules tell the truth?

Derivable ($\vdash$) is about proofs; valid ($\models$) is about execution. Do they agree?

Do the rules tell the truth?

Derivable ($\vdash$) is about proofs; valid ($\models$) is about execution. Do they agree?

Soundness: $\vdash \{P\}\ c\ \{Q\}$ **implies** $\models \{P\}\ c\ \{Q\}$

Check each rule against $\Downarrow$. The rules never lie.

Wrap

Today

- A third semantics: not what a program does, but what it *should* do.
- The Hoare triple $\{P\}\ c\ \{Q\}$, which is partial correctness, so a diverging program satisfies everything.
- Six rules. Five syntax-directed; Conseq is the seam where logic enters.
- The assignment axiom runs backwards and so, therefore, does verification.

Today

- A third semantics: not what a program does, but what it *should* do.
- The Hoare triple $\{P\}\ c\ \{Q\}$, which is partial correctness, so a diverging program satisfies everything.
- Six rules. Five syntax-directed; Conseq is the seam where logic enters.
- The assignment axiom runs backwards and so, therefore, does verification.

Six rules is the *whole* logic. Everything from here is about using them.

Feedback

ATiPL 2026 Minute Sheet

Mid unit feedback